Proposal: A Smarter Long-Term Memory System for AI Assistants

A locally-run prototype using Postgres and Ollama

1. Summary

AI assistants forget things, remember the wrong things, and hold on to facts that are no longer true. Most assistants today store memories as text chunks in a vector database and fetch whatever looks "similar" to the current question. This works for simple cases but breaks down over weeks or months.

This project builds a **memory manager** that behaves more like a human memory: it keeps important things, lets unimportant things fade, notices when new information contradicts old information, updates facts, and condenses old conversations into short summaries. We will compare it against a standard vector-only memory to show it gives better and more consistent answers.

2. The Problem (in plain words)

Imagine you tell an assistant:

- In January: "I live in Turin."
- In June: "I just moved to Milan."
- In October you ask: "What's the weather like where I live?"

A basic vector memory may return *both* statements, or the older one, because both look "similar" to the question. The assistant gets confused or answers wrongly.

Common failures of current systems:

- **Irrelevant recall:** retrieves memories that sound similar but don't matter.
- **Stale facts:** old information is never replaced.
- **No priorities:** "I'm allergic to penicillin" is treated the same as "I had pasta yesterday."
- **No contradiction checks:** conflicting facts sit side by side.
- **Memory bloat:** every message is stored forever, so the memory gets noisy and slow.

3. Goal

Build a prototype memory system that can:

1. Store user memories over long periods.
2. Prioritize the most relevant and important information.
3. Detect contradictory facts.
4. Update outdated information.
5. Compress old history into summaries.
6. Make the assistant's answers more consistent.

4. The Core Idea

Instead of one big pile of text chunks, we store memories in **three layers**, like a simple brain:

Layer	What it holds	Human analogy
Recent memory	Raw recent conversation snippets	Short-term memory
Fact memory	Clean, structured facts ("user lives in Milan") with dates and confidence	Things you *know*
Summary memory	Short digests of older conversations and topics	Fuzzy long-term recollection

A **Memory Manager** sits between the user and the LLM. Before the LLM answers, the manager picks the best few memories. After the LLM answers, the manager decides what is worth saving, updating, or forgetting.

5. How It Works

Step 1: Extract

After each conversation turn, the local LLM reads the exchange and pulls out candidate facts in a fixed format, for example:

`subject: user | relation: lives_in | value: Milan | importance: 0.6 | date: 2026-06-10`

Small talk ("thanks!", "ok") is ignored.

Step 2: Compare with what we already know

For each new fact, we look for existing facts about the same subject and relation (e.g., everything about where the user lives). Then one of four things happens:

- **New:** nothing similar exists, so we save it.
- **Duplicate:** already known, so we just strengthen confidence.
- **Update:** the new value replaces the old one. The old fact is not deleted; it is marked as *superseded* with an end date, so history is preserved.
- **Conflict:** unclear which is right (e.g., two different jobs). We flag it, lower confidence, and optionally let the assistant ask the user to confirm.

Step 3: Retrieve smartly

When the user asks something, we don't rely on similarity alone. Each memory gets a **score**:

`score = similarity × 0.4 + importance × 0.25 + recency × 0.2 + usage × 0.1 + confidence × 0.05`

(The weights are starting values and will be tuned during testing.)

Only current, non-superseded facts are returned by default. The top few memories are placed into the prompt, which keeps it short and focused.

Step 4: Compress

A background job runs periodically (for example, nightly). It:

- Groups old, low-importance conversation snippets by topic.
- Asks the local LLM to write a short summary.
- Saves the summary and archives the originals.

Important facts (health, family, strong preferences) are never summarized away; they stay as structured facts.

Step 5: Forget gently

Memories that are old, unimportant, and never used lose strength over time (like a fading curve). Below a threshold they are archived, not erased, so nothing is lost permanently.

6. How Each Goal Is Met

Goal	How we achieve it
Store memories long-term	Postgres database, three memory layers
Prioritize relevant info	Combined score (similarity + importance + recency + usage)

Goal	How we achieve it
Detect contradictions	Compare new facts against existing facts with the same subject and relation
Update outdated info	Supersede old facts with start/end dates instead of overwriting
Compress history	Scheduled LLM summarization of old, low-value snippets
Improve consistency	Only current, high-confidence facts reach the prompt

7. Technology Stack (all local and free)

Database: Postgres (recommended) with the pgvector extension

- Handles structured facts, dates, and relationships with ordinary tables.
- pgvector adds similarity search in the same database, so you don't need a separate vector store.
- Simple "knowledge graph" links can be stored as a relationships table, so no separate graph database is required for the prototype.

Why not MongoDB? It would work, but Postgres gives you transactions, easy joins between facts and sources, and vector search in one place, which makes contradiction checks and versioning simpler. If you prefer MongoDB, the same design applies using a facts collection plus Atlas-style vector search, but local setup is a bit more awkward.

Local LLM: Ollama

- Chat and extraction model: a 7B to 8B model such as Llama 3.1 8B or Qwen 2.5 7B. A smaller model (3B) can be used if your hardware is limited.
- Embedding model: `nomic-embed-text` or `bge-m3` via Ollama.
- Hardware guide: 16 GB RAM is enough for small models; a GPU with 8 GB or more of VRAM makes it much faster.

Application code

- Python with FastAPI for the memory service.
- A small chat interface (Streamlit or a simple web page) for demos.
- APScheduler or a cron job for compression and decay.

8. Simple Data Design

A minimal set of Postgres tables:

- **messages**: raw conversation snippets (text, timestamp, embedding).
- **facts**: subject, relation, value, importance, confidence, valid_from, valid_to, status (active, superseded, conflicted), source message, embedding.
- **summaries**: topic, summary text, covered date range, embedding, list of source messages.
- **relations**: links between entities (user → family member, user → employer) for lightweight knowledge-graph queries.
- **memory_events**: a log of every add, update, conflict, and archive action, useful for debugging and for your evaluation.

9. System Flow

1. The user sends a message.
2. The Memory Manager retrieves top-ranked facts and summaries.
3. The prompt is built: system instructions, retrieved memories, recent chat, and the new message.
4. The local LLM replies.
5. The extractor reads the exchange, finds new facts, and runs the compare and update step.
6. In the background, compression and decay jobs keep the memory tidy.

10. Evaluation Plan: Proving It Works

The expected outcome is a system that beats traditional vector-only memory. To show this fairly:

Baseline: the same LLM with a plain vector store (store every chunk, retrieve top-k by similarity).

Test data: write or generate 20 to 50 simulated user "life stories" spanning many sessions, including deliberate traps such as moves, job changes, changed preferences, and corrections.

Measures:

- **Fact accuracy:** does the assistant give the current, correct answer?
- **Staleness rate:** how often does it use outdated information?
- **Contradiction detection rate:** how many planted contradictions are caught?

- **Retrieval precision:** how many retrieved memories were actually useful?
- **Prompt size and latency:** tokens used and response time.
- **Consistency:** ask the same question in different ways; do answers stay the same?
- **Human rating:** a small group scores answers from 1 to 5.

Success means the new system scores clearly better on accuracy, staleness, and consistency, while using fewer prompt tokens.

11. Project Timeline (about 10 weeks)

Weeks	Work
1	Literature review (e.g., MemGPT, Generative Agents, Mem0, Zep), finalize design
2	Set up Postgres, pgvector, and Ollama; build the baseline vector memory
3-4	Fact extraction and storage
5	Contradiction detection and update logic
6	Scoring and smart retrieval
7	Compression and decay jobs
8	Chat interface and full integration
9	Build test scenarios and run experiments
10	Analyze results, tune weights, write the report

12. Risks and How to Handle Them

- **Small local models extract facts badly.** Use strict output formats (JSON), few-shot examples, and validation; try a larger model for extraction only.
- **False contradictions** (e.g., someone genuinely has two hobbies). Mark relations as *single-valued* (home city) or *multi-valued* (hobbies) so only the right ones trigger updates.
- **Summaries lose detail.** Keep originals archived and linked to the summary.
- **Privacy.** Everything runs locally, so no user data leaves the machine. Add a "view and delete my memories" feature.

- **Slow responses.** Run extraction and compression in the background, not before the reply.

13. Expected Outcome

A working prototype that:

- Remembers users across many sessions.
- Keeps facts current and flags conflicts.
- Stays compact through summaries and gentle forgetting.
- Gives measurably more accurate and consistent answers than a standard vector-only memory.

14. Possible Future Work

- A full graph database (e.g., Neo4j) for richer relationship reasoning.
- A user-facing memory dashboard to review and edit what the assistant knows.
- Learning the scoring weights automatically instead of setting them by hand.
- Multi-user support and encrypted storage.